

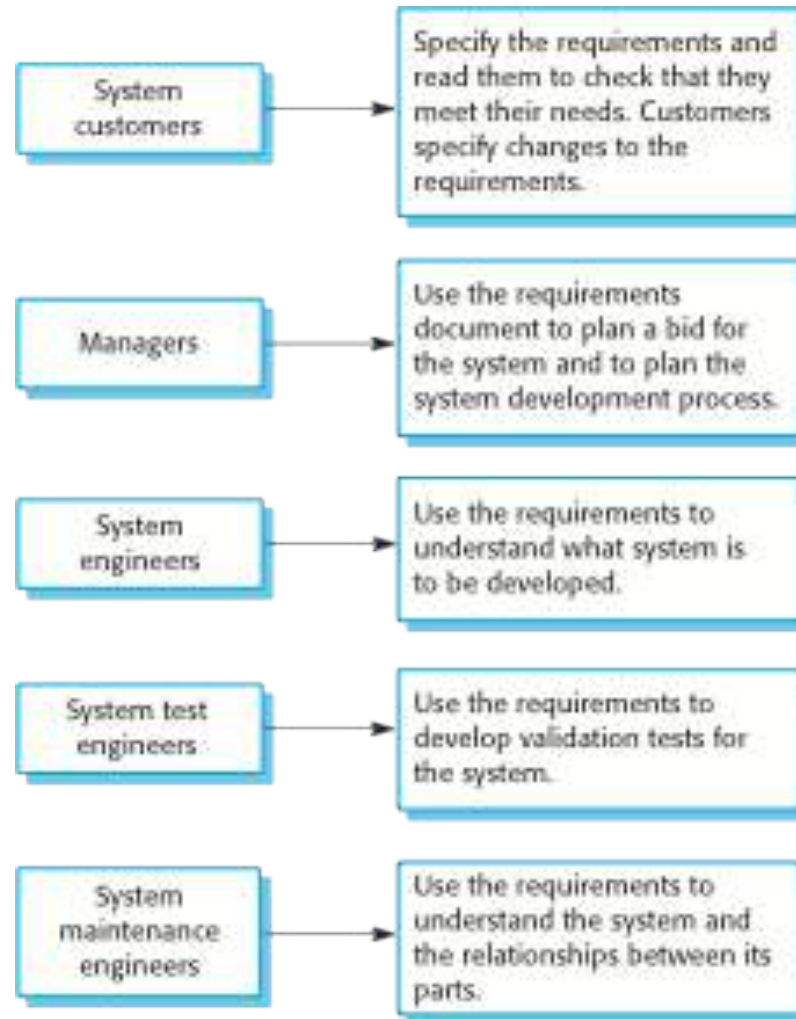
Unit-2

- Requirements engineering process: Feasibility studies, Requirement's elicitation and analysis, Requirements validation, Requirements management.
- System models: Context models, Behavioural models, Data models, Object models, Structured methods.

The software requirements document

- The software requirements document is the official statement of what is required of the system developers.
- Should include both a definition of user requirements and a specification of the system requirements.
- It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.

Users of a requirements document



Requirements document variability

- Information in requirements document depends on type of system and the approach to development used.
- Systems developed incrementally will, typically, have less detail in the requirements document.
- Requirements documents standards have been designed e.g. IEEE standard. These are mostly applicable to the requirements for large systems engineering projects.

The structure of a requirements document

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

Requirements specification

- The process of writing down the user and system requirements in a requirements document.
- User requirements have to be understandable by end-users and customers who do not have a technical background.
- System requirements are more detailed requirements and may include more technical information.
- The requirements may be part of a contract for the system development
 - It is therefore important that these are as complete as possible.

Ways of writing a system requirements specification

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Requirements engineering process

- The **Requirements Engineering (RE) Process** is a structured approach used in software and systems development to **identify, analyze, document, validate, and manage requirements** for a project.

Feasibility studies

- A **feasibility study** is an early step in the **Requirements Engineering process** that evaluates whether a proposed system or project is **practical and worth implementing**.

Types of Feasibility Study

- Technical Feasibility
- Economic Feasibility
- Operational Feasibility
- Legal Feasibility
- Schedule Feasibility

Examples of Feasibility Study

Hospital Management System

Problem:

Manual record-keeping leads to errors, delays, and data loss.

Feasibility Analysis:

Technical Feasibility

Computers, database systems, and cloud are available

Economic Feasibility

Initial cost is high, but reduces paperwork and staff cost

Operational Feasibility

Doctors and staff can be trained easily

Legal Feasibility

Must follow patient data privacy regulations

Schedule Feasibility

Can be implemented in 6–8 months

Conclusion:  Feasible

Blockchain-Based Healthcare System

Problem:

- Secure sharing of patient records across hospitals.

Feasibility Analysis:

- **Technical Feasibility**

Blockchain technology available but complex

- **Economic Feasibility**

High implementation cost

- **Operational Feasibility**

Requires training and adoption by hospitals

- **Legal Feasibility**

Strict healthcare data regulations

- **Schedule Feasibility**

Longer development time

- **Conclusion:** Partially Feasible (needs careful planning)

- An educational institution plans to develop an **E-Learning Platform** that allows students to access study materials, attend live classes, submit assignments, and take online exams.
- Perform a **feasibility study** for the proposed E-Learning Platform. Analyze the following aspects:
 1. Technical Feasibility
 2. Economic Feasibility
 3. Operational Feasibility
 4. Legal Feasibility
 5. Schedule Feasibility

Conclude whether the system is feasible or not with justification.

Main Phases of Requirements Engineering

- Requirements elicitation
- Requirements analysis
- Requirements validation
- Requirements management

Requirements elicitation

- **Requirements elicitation** is the process of **discovering, gathering, and understanding the needs and requirements of stakeholders** for a system.
- According to Software Engineering, elicitation is one of the most critical and challenging activities in Requirements Engineering because:
 - Stakeholders may not know exactly what they want
 - Requirements may be unclear, incomplete, or conflicting
 - Communication gaps can occur between users and developers

Objectives of Requirements Elicitation

- Identify stakeholder needs
- Understand the problem domain
- Capture both **functional** and **non-functional** requirements
- Define system boundaries

Elicitation Techniques

- Interviews
- Questionnaires / Surveys
- Observation (Ethnography)
- Scenarios & Use Cases
- Prototyping
- Workshops / Joint Application Development

Requirements Analysis

- **Requirements Analysis** is the process of **examining, refining, organizing, and structuring the requirements** gathered during elicitation

Objectives of Requirements Analysis

- Convert raw requirements into **clear and structured form**
- Identify **conflicts and ambiguities**
- Classify requirements (functional & non-functional)
- Check feasibility and constraints
- Prepare for specification (SRS)

Key Activities in Requirements Analysis

Classification:

Requirements are divided into functional and non-functional

Organization:

Group related requirements into modules

Prioritization:

Identify important requirements using techniques.

Conflict Resolution:

Resolve disagreements between stakeholders

Feasibility Check:

Ensure requirements are realistic and achievable

Modeling:

Represent requirements using diagrams like Use Case and DFD

Example (E-Learning Platform)

- Raw Requirement:
“System should help students learn”

After Analysis:

- Students can register and login
- Attend live classes
- Access recorded videos
- Submit assignments

The requirement becomes **clear and implementable**.

Requirements validation

- **Requirements Validation** is the process of checking whether the documented requirements are correct, complete, consistent, and reflect the actual needs of stakeholders.

Objectives of Requirements Validation

- Ensure requirements match stakeholder expectations
- Detect errors early (before design & coding)
- Verify completeness and consistency
- Improve requirement quality

Techniques for Requirements Validation

- Requirements Reviews
- Prototyping
- Test Case Generation
- Automated Consistency Checking

Requirements Management

- **Requirements Management** is the process of handling, tracking, and controlling changes to requirements throughout the software development lifecycle.

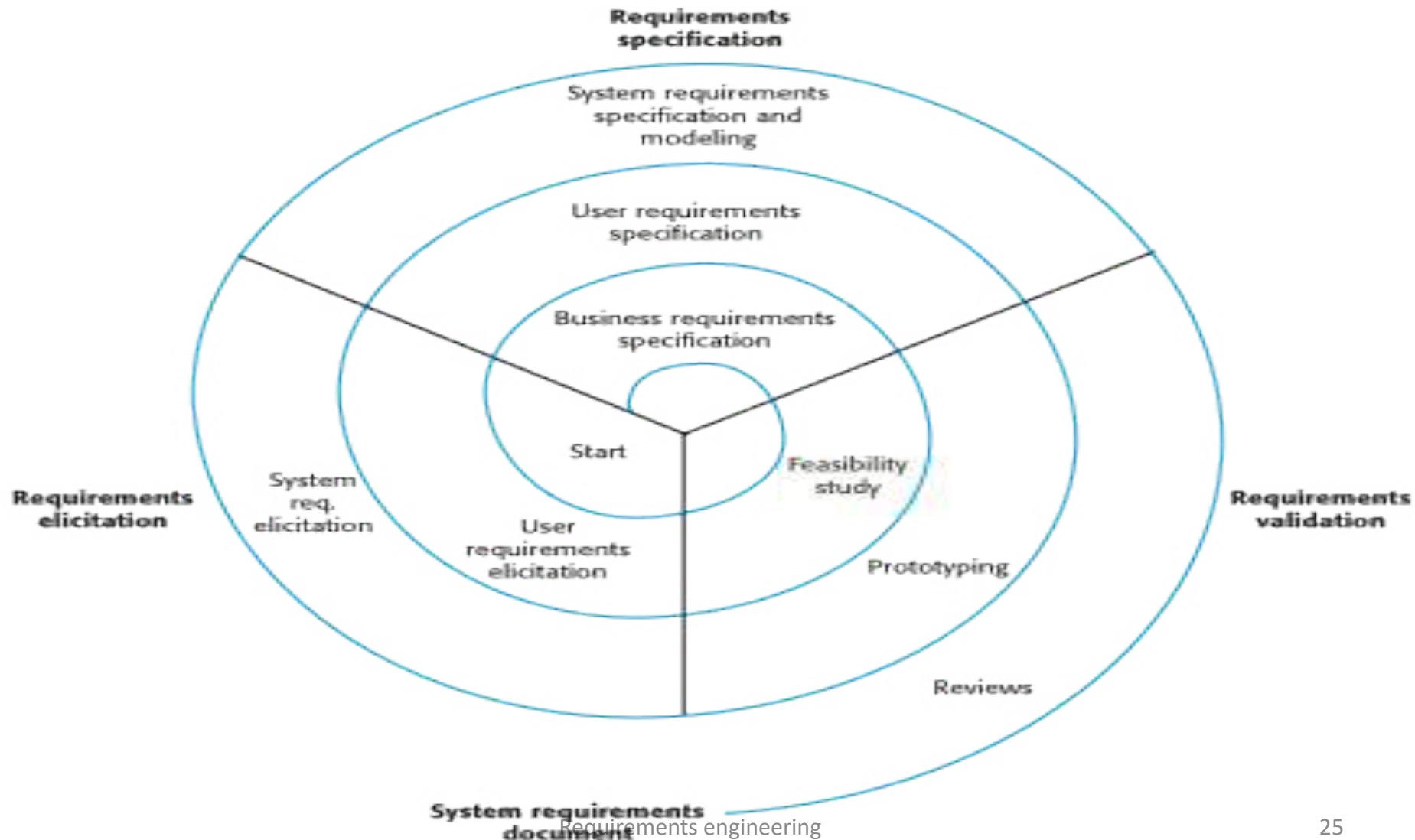
Objectives of Requirements Management

- Manage changing requirements effectively
- Maintain **consistency and traceability**
- Control scope changes (avoid scope creep)
- Ensure all requirements are implemented correctly

Key Activities in Requirements Management

- Change Management
- Requirements Traceability
- Version Control
- Status Tracking

A spiral view of the requirements engineering process



Requirements elicitation and analysis

- Sometimes called requirements elicitation or requirements discovery.
- Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints.
- May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called *stakeholders*.

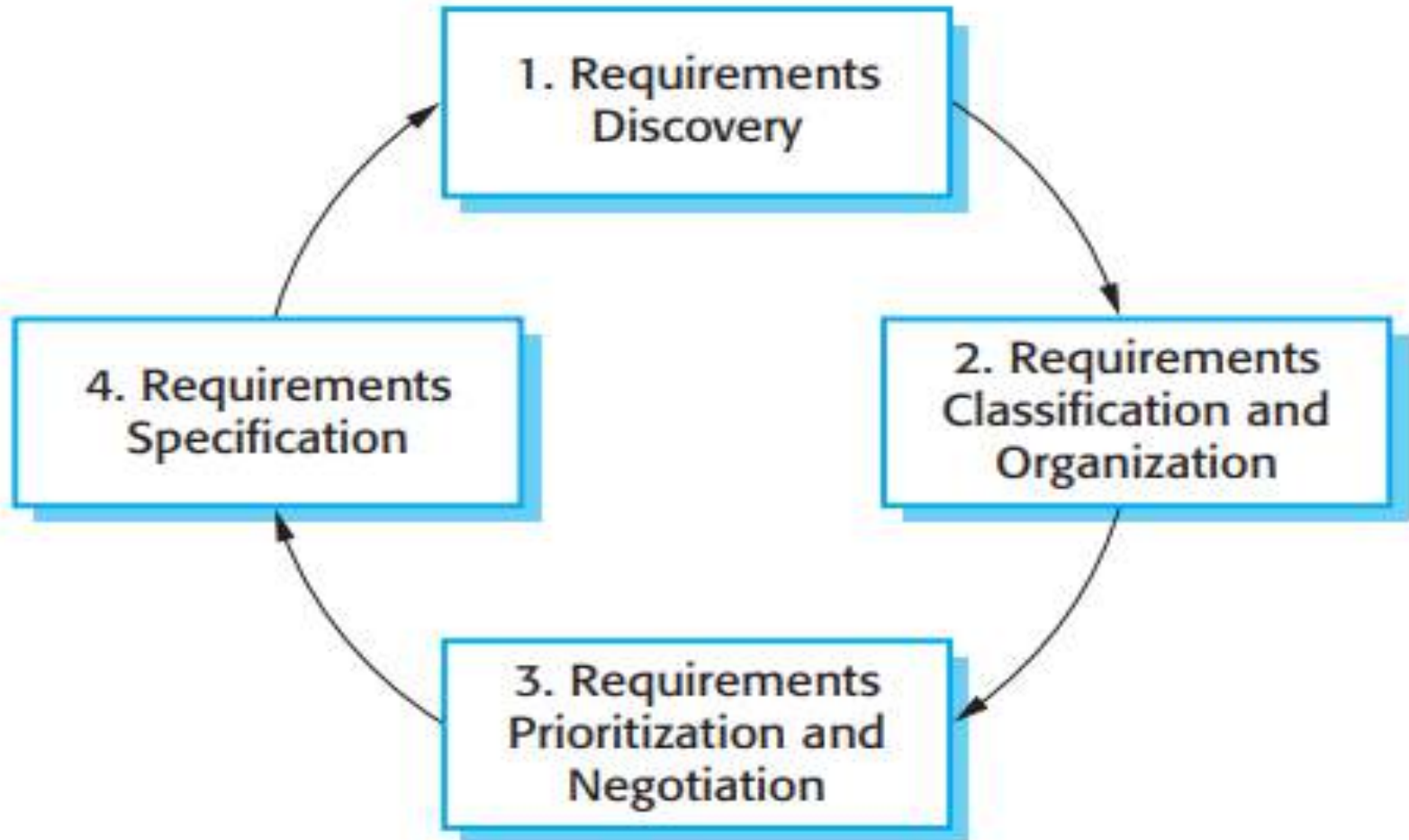
Problems of requirements elicitation

- Stakeholders don't know what they really want.
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment change.

Requirements elicitation and analysis

- Software engineers work with a range of system stakeholders to find out about the application domain, the services that the system should provide, the required system performance, hardware constraints, other systems, etc.
- Stages include:
 - Requirements discovery,
 - Requirements classification and organization,
 - Requirements prioritization and negotiation,
 - Requirements specification.

The requirements elicitation and analysis process



Process activities

- Requirements discovery
 - Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- Requirements classification and organisation
 - Groups related requirements and organises them into coherent clusters.
- Prioritisation and negotiation
 - Prioritising requirements and resolving requirements conflicts.
- Requirements specification
 - Requirements are documented and input into the next round of the spiral.

Sources of Requirements

- Stakeholders
- Existing softwares
- Process and Policy Documents
- Domain
- System

Stakeholders in the MHC-PMS

- Patients whose information is recorded in the system.
- Doctors who are responsible for assessing and treating patients.
- Nurses who coordinate the consultations with doctors and administer some treatments.
- Medical receptionists who manage patients' appointments.
- IT staff who are responsible for installing and maintaining the system.

Stakeholders in the MHC-PMS

- A medical ethics manager who must ensure that the system meets current ethical guidelines for patient care.
- Health care managers who obtain management information from the system.
- Medical records staff who are responsible for ensuring that system information can be maintained and preserved, and that record keeping procedures have been properly implemented.

A **viewpoint** is a perspective used to examine a system's requirements, usually representing a specific stakeholder or source of requirements (like users, managers, or external systems).

Viewpoints = different ways of looking at a system

Each person sees the system differently based on their role.

Types of Viewpoints

Interactor Viewpoints

- Represent **people or systems that directly interact** with the system
- Examples:
 - Customers using an ATM
 - External systems (e.g., database)

Focus: **System functionality and interactions**

Indirect Viewpoints

- Stakeholders who **do not directly use the system** but influence requirements
- Examples:
 - Managers
 - Security staff
 - Regulatory authorities

Focus: **Policies, constraints, and organizational needs**

Domain Viewpoints

- Represent **domain-specific rules, standards, or constraints**
- Examples:
 - Banking regulations
 - Industry standards

Focus: **Environment and domain constraints**

Example: Online Shopping System

1. Interactor Viewpoint (Direct users)

Customer

- Search products
- Add to cart
- Make payment
- Track orders

Seller

- Upload products
- Manage prices
- View orders

Indirect Viewpoint (Not direct users)

Company Manager

- Check sales reports
- Monitor performance
- Plan offers

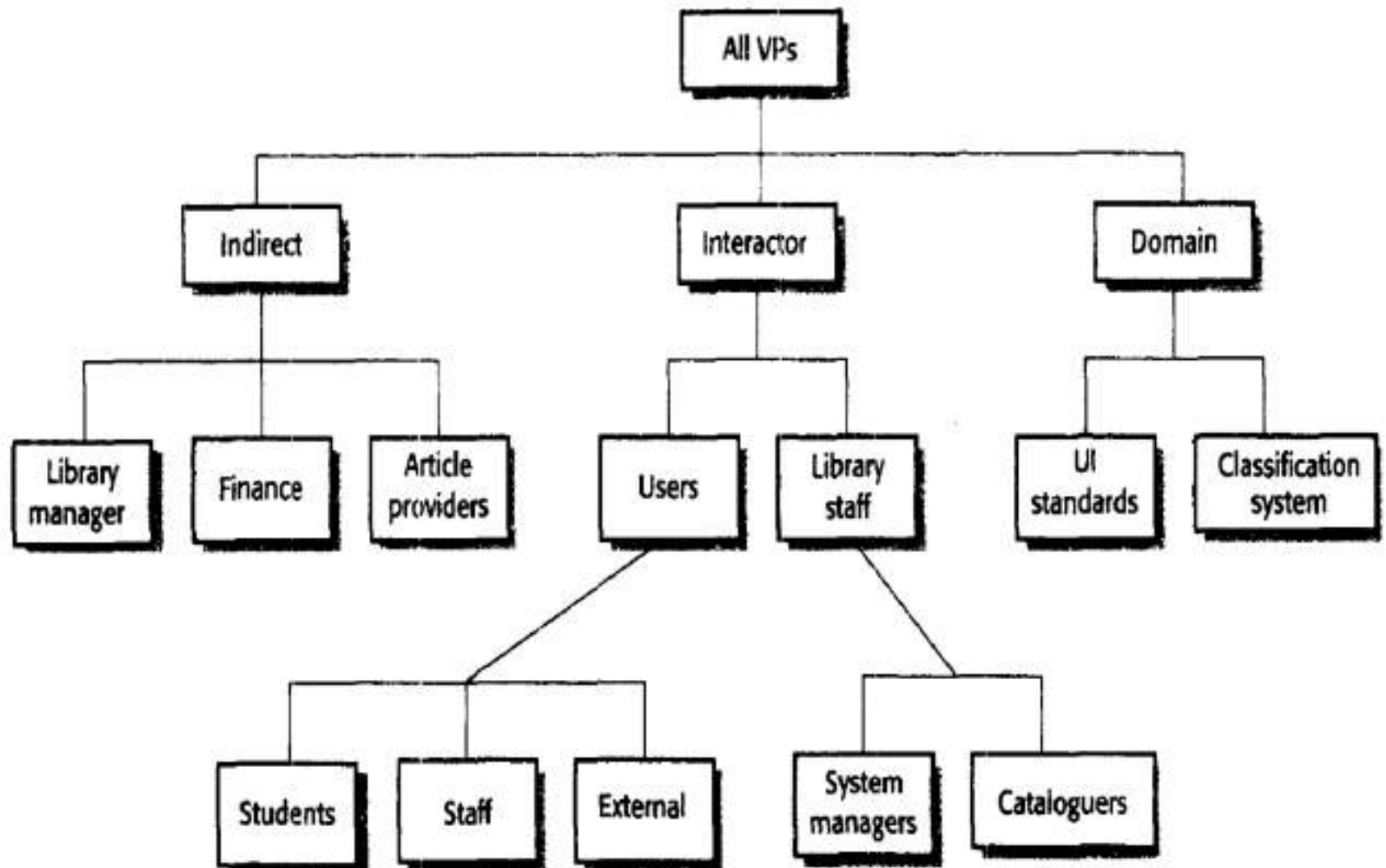
Delivery Staff

- Deliver products on time
- Update delivery status

Domain Viewpoint (Rules & constraints)

E-commerce Rules / Laws

- Secure online payments
- Return/refund policies
- Customer data protection



Ethnography

- Ethnography is a **qualitative research method** used to study people, cultures, and behaviors in their **natural environment**.
- “Ethnography means observing and understanding how people behave in real-life settings.”

Key Features of Ethnography

- Conducted in **natural settings** (not lab)
- Focus on **people, culture, behavior**
- Uses **observation + interaction**
- Produces **detailed descriptions**
- Researcher may become part of the group

Types of Ethnography

1. Participant Observation

- Researcher actively participates
- Example: Joining a classroom as a student

2. Non-Participant Observation

- Researcher observes without involvement
- Example: Watching students silently

3. Digital Ethnography (Netnography)

- Study of online communities
- Example: Analyzing student discussions

Interviews in practice

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviews are not good for understanding domain requirements
 - Requirements engineers cannot understand specific domain terminology;
 - Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.

Pros and Cons

- **Advantages:**

- Cost involved is very less
- Good to get an overall understanding of the system.

Disadvantages:

- It is difficult to get organisational requirements and constraints.
- May result in a lot of useless conversations. Hence it is recommended to use other requirements elicitation techniques

Scenarios

- Scenarios are real-life examples of how a system can be used.
- They should include
 - A description of the starting situation;
 - A description of the normal flow of events;
 - A description of what can go wrong;
 - Information about other concurrent activities;
 - A description of the state when the scenario finishes.

Scenario for collecting medical history in MHC-PMS

Initial assumption: The patient has seen a medical receptionist who has created a record in the system and collected the patient's personal information (name, address, age, etc.). A nurse is logged on to the system and is collecting medical history.

Normal: The nurse searches for the patient by family name. If there is more than one patient with the same surname, the given name (first name in English) and date of birth are used to identify the patient.

The nurse chooses the menu option to add medical history.

The nurse then follows a series of prompts from the system to enter information about consultations elsewhere on mental health problems (free text input), existing medical conditions (nurse selects conditions from menu), medication currently taken (selected from menu), allergies (free text), and home life (form).

Scenario for collecting medical history in MHC-PMS

What can go wrong: The patient's record does not exist or cannot be found. The nurse should create a new record and record personal information.

Patient conditions or medication are not entered in the menu. The nurse should choose the 'other' option and enter free text describing the condition/medication.

Patient cannot/will not provide information on medical history. The nurse should enter free text recording the patient's inability/unwillingness to provide information. The system should print the standard exclusion form stating that the lack of information may mean that treatment will be limited or delayed. This should be signed and handed to the patient.

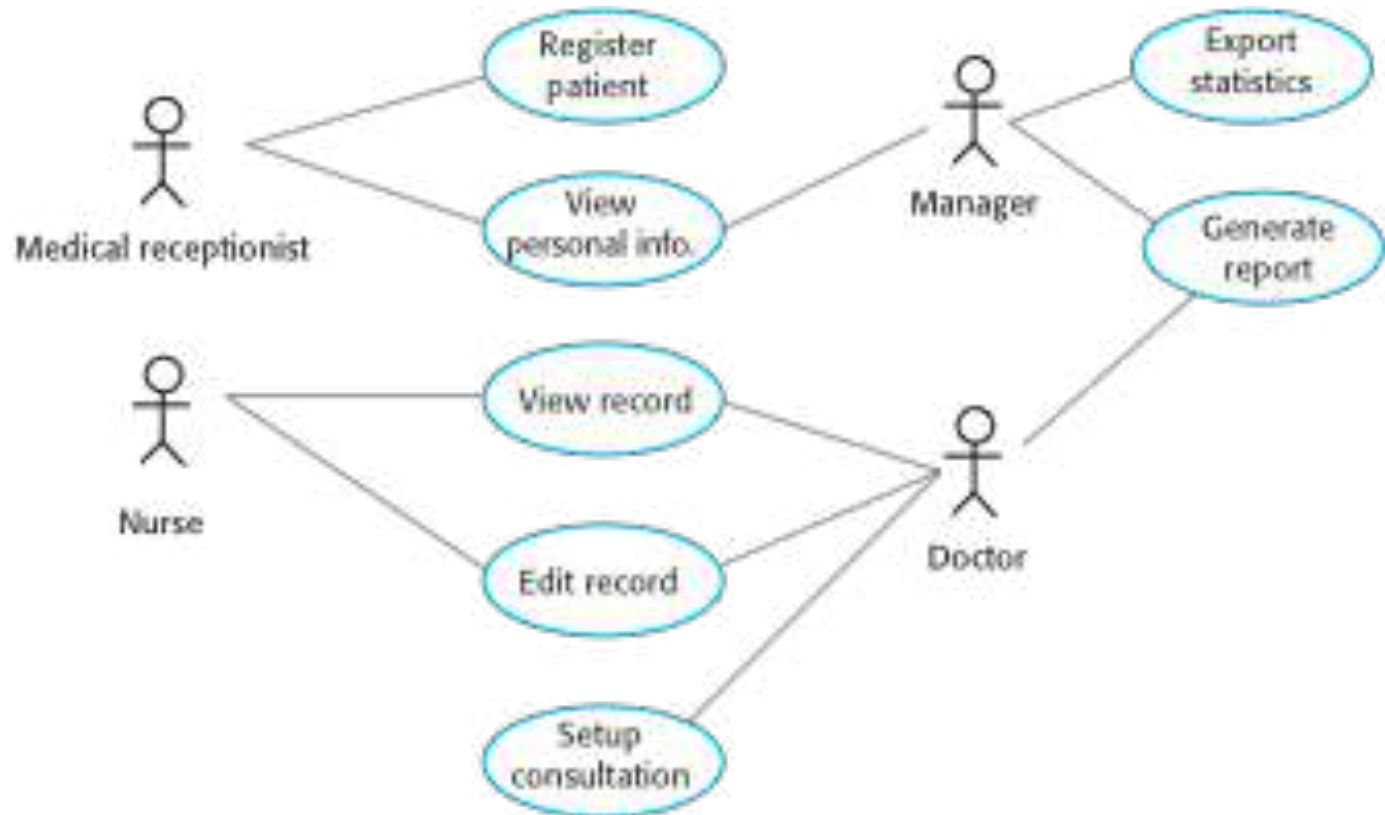
Other activities: Record may be consulted but not edited by other staff while information is being entered.

System state on completion: User is logged on. The patient record including medical history is entered in the database, a record is added to the system log showing the start and end time of the session and the nurse involved.

Use cases

- Use-cases are a scenario based technique in the UML which identify the actors in an interaction and which describe the interaction itself.
- A set of use cases should describe all possible interactions with the system.
- High-level graphical model supplemented by more detailed tabular description (see Chapter 5).
- Sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing in the system.

Use cases for the MHC-PMS



Ethnography

- A social scientist spends a considerable time observing and analysing how people actually work.
- People do not have to explain or articulate their work.
- Social and organisational factors of importance may be observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

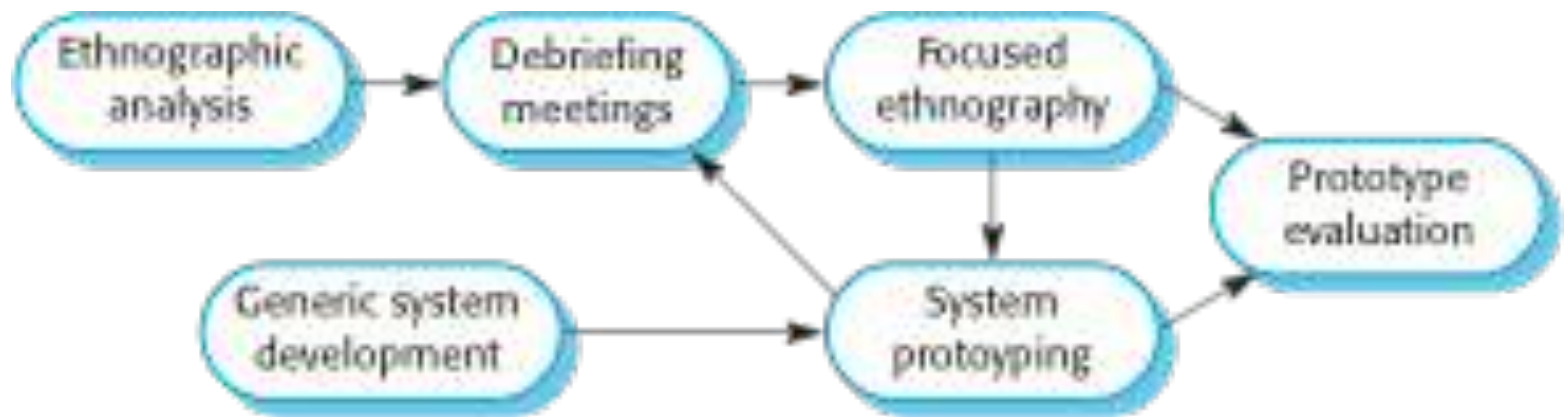
Scope of ethnography

- Requirements that are derived from the way that people actually work rather than the way in which process definitions suggest that they ought to work.
- Requirements that are derived from cooperation and awareness of other people's activities.
 - Awareness of what other people are doing leads to changes in the ways in which we do things.
- Ethnography is effective for understanding existing processes but cannot identify new features that should be added to a system.

Focused ethnography

- Developed in a project studying the air traffic control process
- Combines ethnography with prototyping
- Prototype development results in unanswered questions which focus the ethnographic analysis.
- The problem with ethnography is that it studies existing practices which may have some historical basis which is no longer relevant.

Ethnography and prototyping for requirements analysis



Requirements validation

- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important
 - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking

- **Validity**. Does the system provide the functions which best support the customer's needs?
- **Consistency**. Are there any requirements conflicts?
- **Completeness**. Are all functions required by the customer included?
- **Realism**. Can the requirements be implemented given available budget and technology
- **Verifiability**. Can the requirements be checked?

Requirements validation techniques

- Requirements reviews
 - Systematic manual analysis of the requirements.
- Prototyping
 - Using an executable model of the system to check requirements. Covered in Chapter 2.
- Test-case generation
 - Developing tests for requirements to check testability.

Requirements reviews

- Regular reviews should be held while the requirements definition is being formulated.
- Both client and contractor staff should be involved in reviews.
- Reviews may be formal (with completed documents) or informal. Good communications between developers, customers and users can resolve problems at an early stage.

Review checks

- **Verifiability**
 - Is the requirement realistically testable?
- **Comprehensibility**
 - Is the requirement properly understood?
- **Traceability**
 - Is the origin of the requirement clearly stated?
- **Adaptability**
 - Can the requirement be changed without a large impact on other requirements?

Requirements management

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development.
- New requirements emerge as a system is being developed and after it has gone into use.
- You need to keep track of individual requirements and maintain links between dependent requirements so that you can assess the impact of requirements changes. You need to establish a formal process for making change proposals and linking these to system requirements.

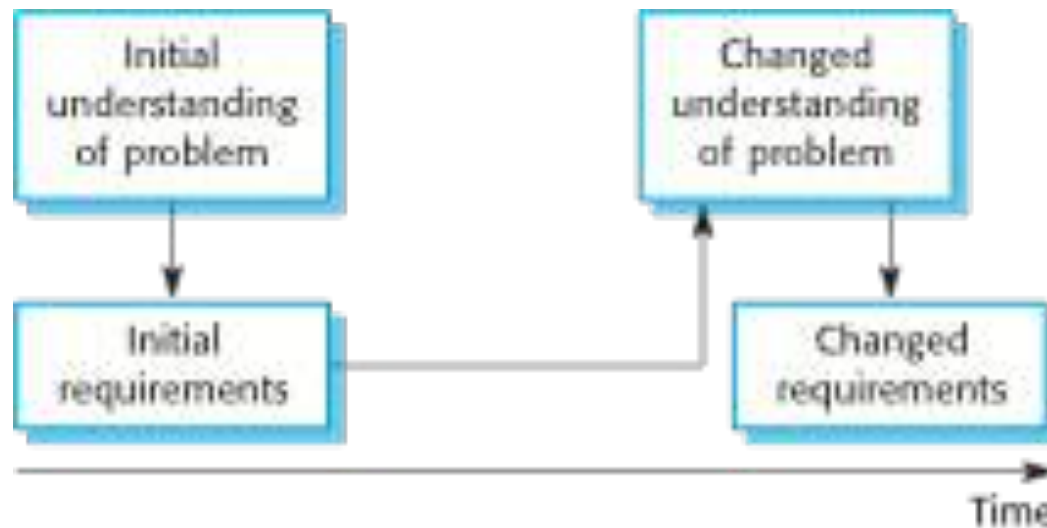
Changing requirements

- The business and technical environment of the system always changes after installation.
 - New hardware may be introduced, it may be necessary to interface the system with other systems, business priorities may change (with consequent changes in the system support required), and new legislation and regulations may be introduced that the system must necessarily abide by.
- The people who pay for a system and the users of that system are rarely the same people.
 - System customers impose requirements because of organizational and budgetary constraints. These may conflict with end-user requirements and, after delivery, new features may have to be added for user support if the system is to meet its goals.

Changing requirements

- Large systems usually have a diverse user community, with many users having different requirements and priorities that may be conflicting or contradictory.
 - The final system requirements are inevitably a compromise between them and, with experience, it is often discovered that the balance of support given to different users has to be changed.

Requirements evolution



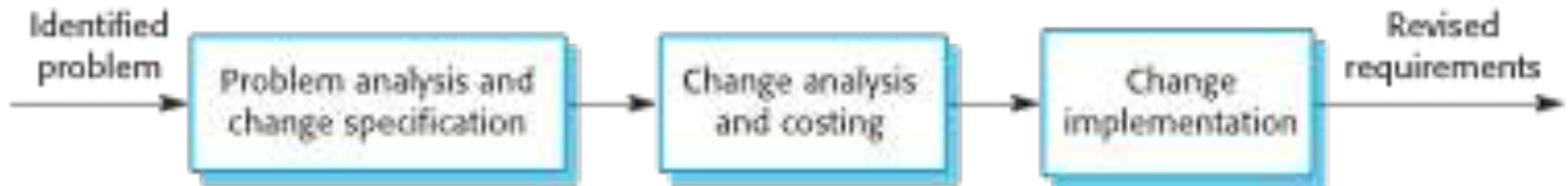
Requirements management planning

- Establishes the level of requirements management detail that is required.
- Requirements management decisions:
 - *Requirements identification* Each requirement must be uniquely identified so that it can be cross-referenced with other requirements.
 - *A change management process* This is the set of activities that assess the impact and cost of changes. I discuss this process in more detail in the following section.
 - *Traceability policies* These policies define the relationships between each requirement and between the requirements and the system design that should be recorded.
 - *Tool support* Tools that may be used range from specialist requirements management systems to spreadsheets and simple database systems.

Requirements change management

- Deciding if a requirements change should be accepted
 - *Problem analysis and change specification*
 - During this stage, the problem or the change proposal is analyzed to check that it is valid. This analysis is fed back to the change requestor who may respond with a more specific requirements change proposal, or decide to withdraw the request.
 - *Change analysis and costing*
 - The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.
 - *Change implementation*
 - The requirements document and, where necessary, the system design and implementation, are modified. Ideally, the document should be organized so that changes can be easily implemented.

Requirements change management



Key points

- You can use a range of techniques for requirements elicitation including interviews, scenarios, use-cases and ethnography.
- Requirements validation is the process of checking the requirements for validity, consistency, completeness, realism and verifiability.
- Business, organizational and technical changes inevitably lead to changes to the requirements for a software system. Requirements management is the process of managing and controlling these changes.



Chapter 5 – System Modeling

Topics covered



- ✧ Context models
- ✧ Interaction models
- ✧ Structural models
- ✧ Behavioral models
- ✧ Model-driven engineering

System modeling



- ✧ System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- ✧ System modeling has now come to mean representing a system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).
- ✧ System modelling helps the analyst to understand the functionality of the system and models are used to communicate with customers.

Existing and planned system models



- ✧ Models of the existing system are used during requirements engineering. They help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system.
- ✧ Models of the new system are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.
- ✧ In a model-driven engineering process, it is possible to generate a complete or partial system implementation from the system model.

System perspectives



- ✧ An external perspective, where you model the context or environment of the system.
- ✧ An interaction perspective, where you model the interactions between a system and its environment, or between the components of a system.
- ✧ A structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- ✧ A behavioral perspective, where you model the dynamic behavior of the system and how it responds to events.

UML diagram types



- ✧ Activity diagrams, which show the activities involved in a process or in data processing .
- ✧ Use case diagrams, which show the interactions between a system and its environment.
- ✧ Sequence diagrams, which show interactions between actors and the system and between system components.
- ✧ Class diagrams, which show the object classes in the system and the associations between these classes.
- ✧ State diagrams, which show how the system reacts to internal and external events.

Use of graphical models



- ✧ As a means of facilitating discussion about an existing or proposed system
 - Incomplete and incorrect models are OK as their role is to support discussion.
- ✧ As a way of documenting an existing system
 - Models should be an accurate representation of the system but need not be complete.
- ✧ As a detailed system description that can be used to generate a system implementation
 - Models have to be both correct and complete.



Context models

Context models



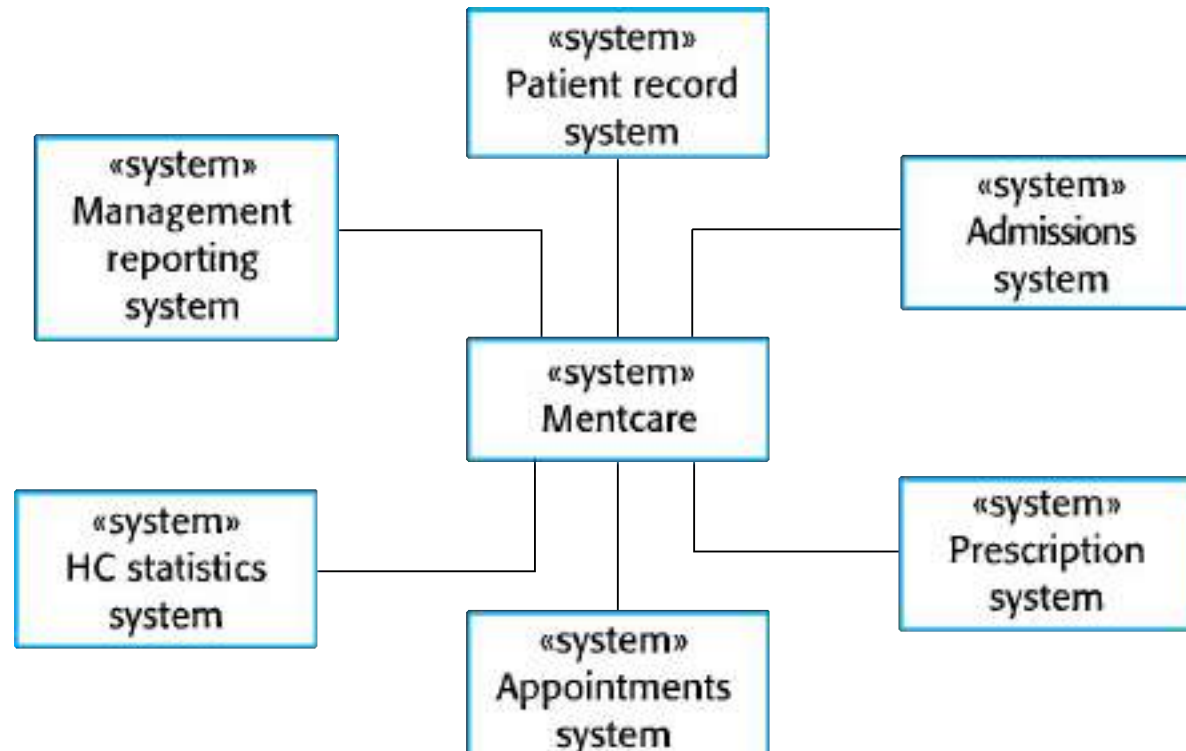
- ✧ Context models are used to illustrate the operational context of a system - they show what lies outside the system boundaries.
- ✧ Social and organisational concerns may affect the decision on where to position system boundaries.
- ✧ Architectural models show the system and its relationship with other systems.

System boundaries



- ✧ System boundaries are established to define what is inside and what is outside the system.
 - They show other systems that are used or depend on the system being developed.
- ✧ The position of the system boundary has a profound effect on the system requirements.
- ✧ Defining a system boundary is a political judgment
 - There may be pressures to develop system boundaries that increase / decrease the influence or workload of different parts of an organization.

The context of the Mentcare system

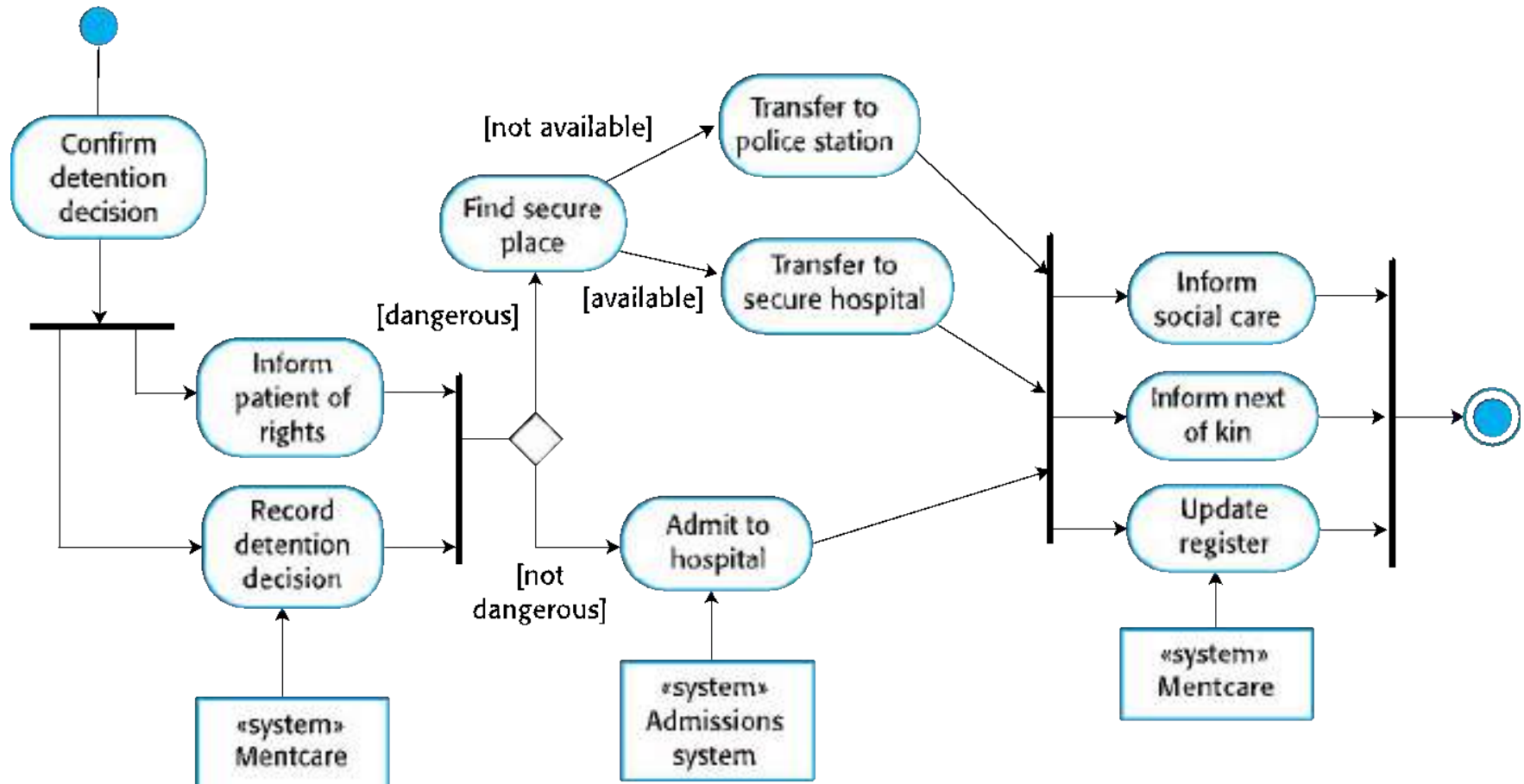


Process perspective



- ✧ Context models simply show the other systems in the environment, not how the system being developed is used in that environment.
- ✧ Process models reveal how the system being developed is used in broader business processes.
- ✧ UML activity diagrams may be used to define business process models.

Process model of involuntary detention





Interaction models

Interaction models



- ✧ Modeling user interaction is important as it helps to identify user requirements.
- ✧ Modeling system-to-system interaction highlights the communication problems that may arise.
- ✧ Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.
- ✧ Use case diagrams and sequence diagrams may be used for interaction modeling.

Use case modeling



- ✧ Use cases were developed originally to support requirements elicitation and now incorporated into the UML.
- ✧ Each use case represents a discrete task that involves external interaction with a system.
- ✧ Actors in a use case may be people or other systems.
- ✧ Represented diagrammatically to provide an overview of the use case and in a more detailed textual form.

Transfer-data use case



✧ A use case in the Mentcare system

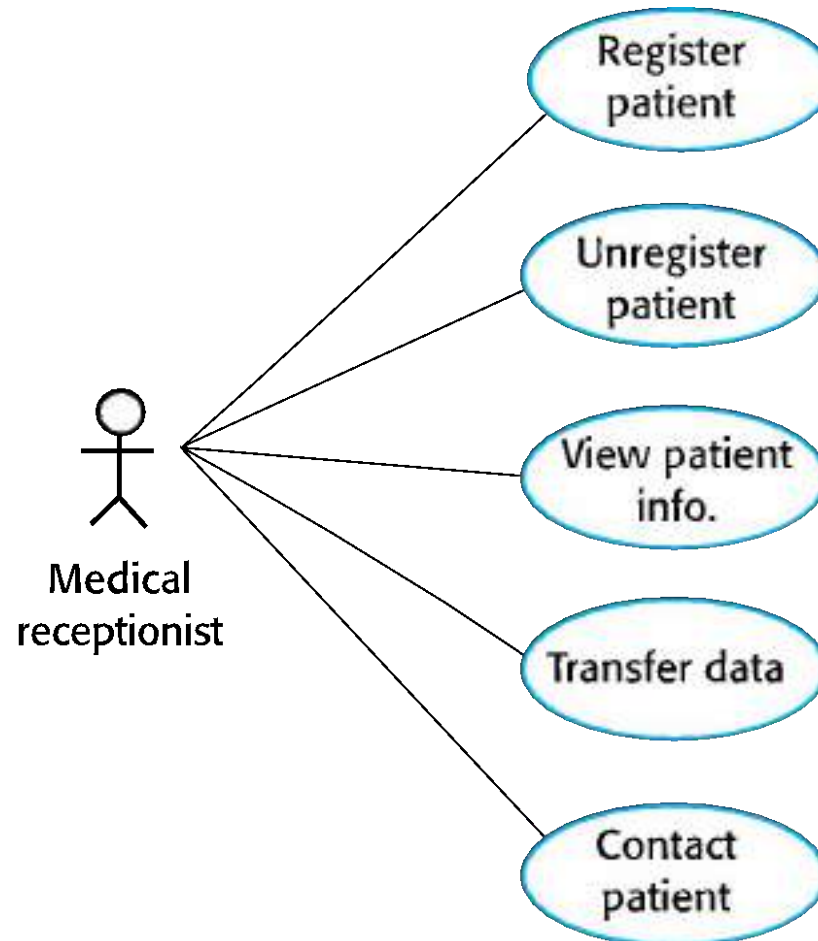


Tabular description of the 'Transfer data' use-case



MHC-PMS: Transfer data	
Actors	Medical receptionist, patient records system (PRS)
Description	A receptionist may transfer data from the Mentcase system to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary
Stimulus	User command issued by medical receptionist
Response	Confirmation that PRS has been updated
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

Use cases in the Mentcare system involving the role 'Medical Receptionist'



Sequence diagrams

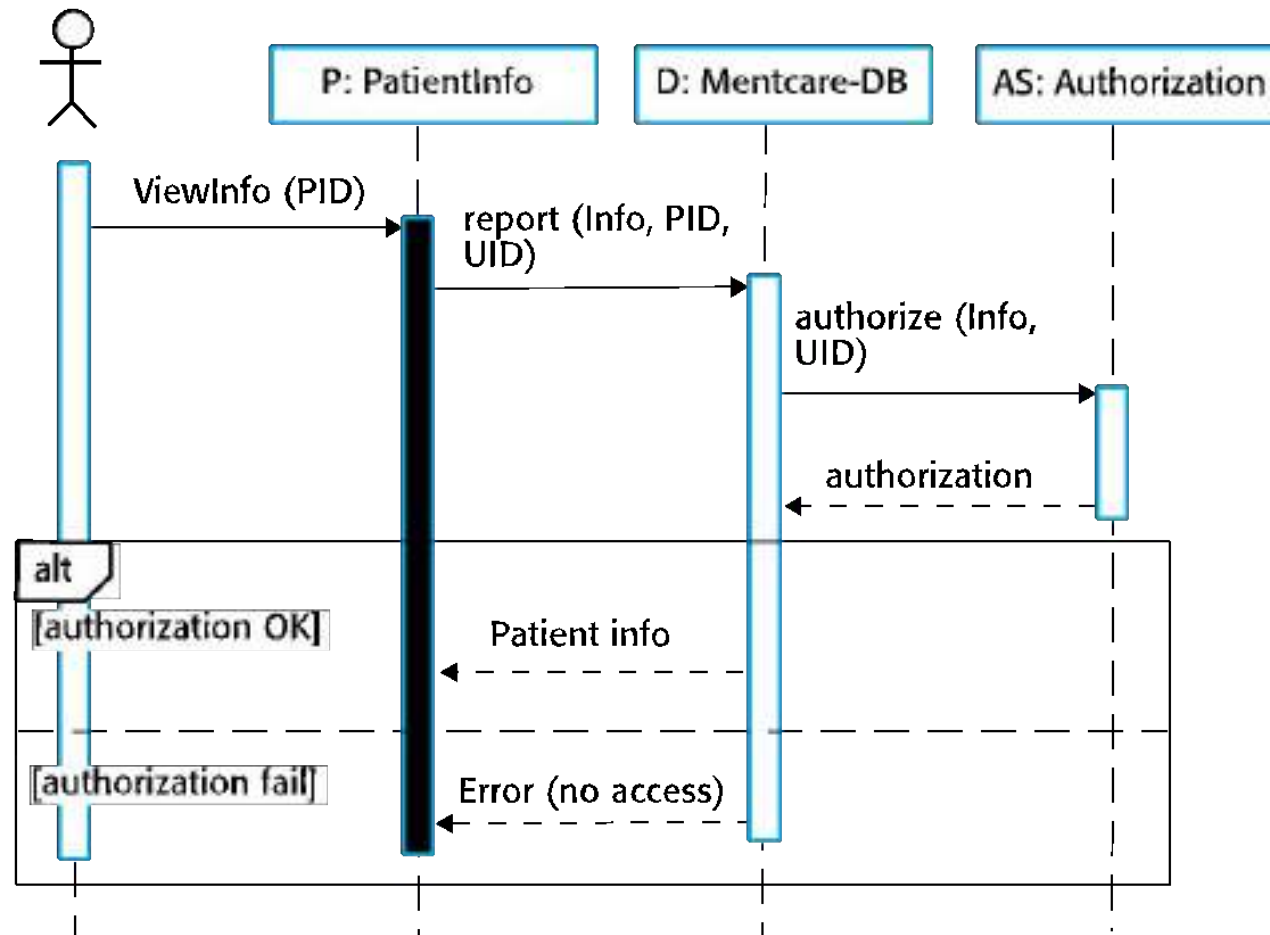


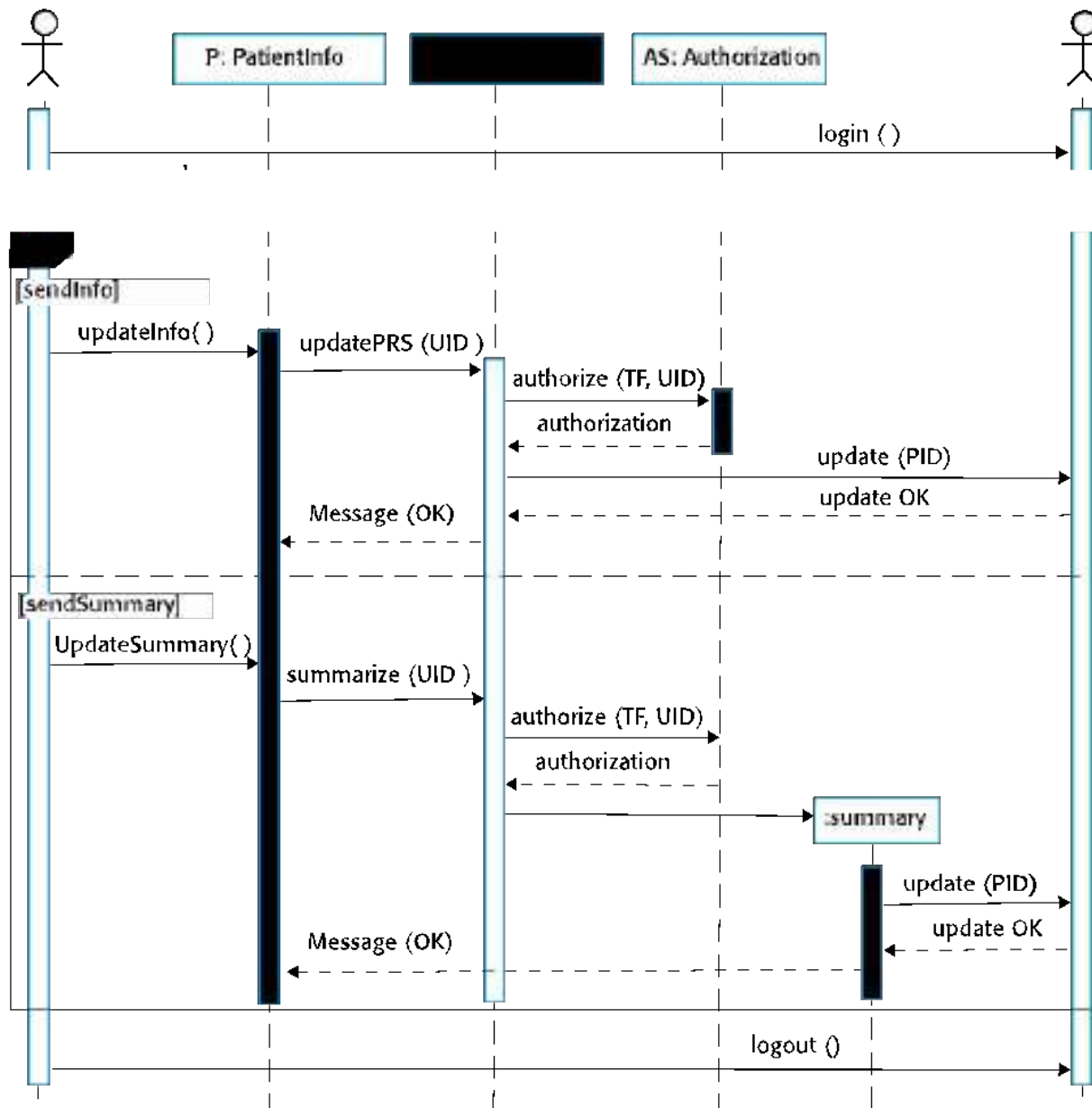
- ✧ Sequence diagrams are part of the UML and are used to model the interactions between the actors and the objects within a system.
- ✧ A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance.
- ✧ The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these.
- ✧ Interactions between objects are indicated by annotated arrows.

Sequence diagram for View patient information



Medical Receptionist





Sequence diagram for Transfer Data



Structural models

Structural models



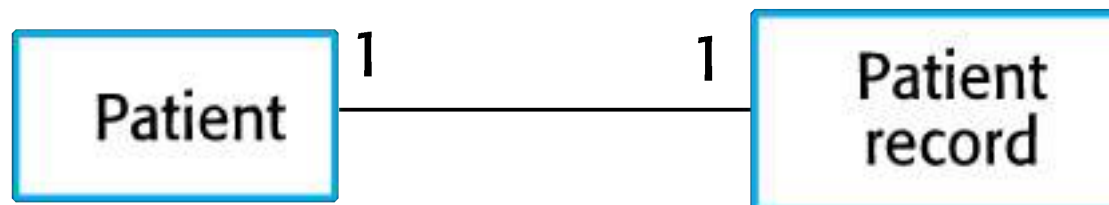
- ✧ Structural models of software display the organization of a system in terms of the components that make up that system and their relationships.
- ✧ Structural models may be static models, which show the structure of the system design, or dynamic models, which show the organization of the system when it is executing.
- ✧ You create structural models of a system when you are discussing and designing the system architecture.

Class diagrams

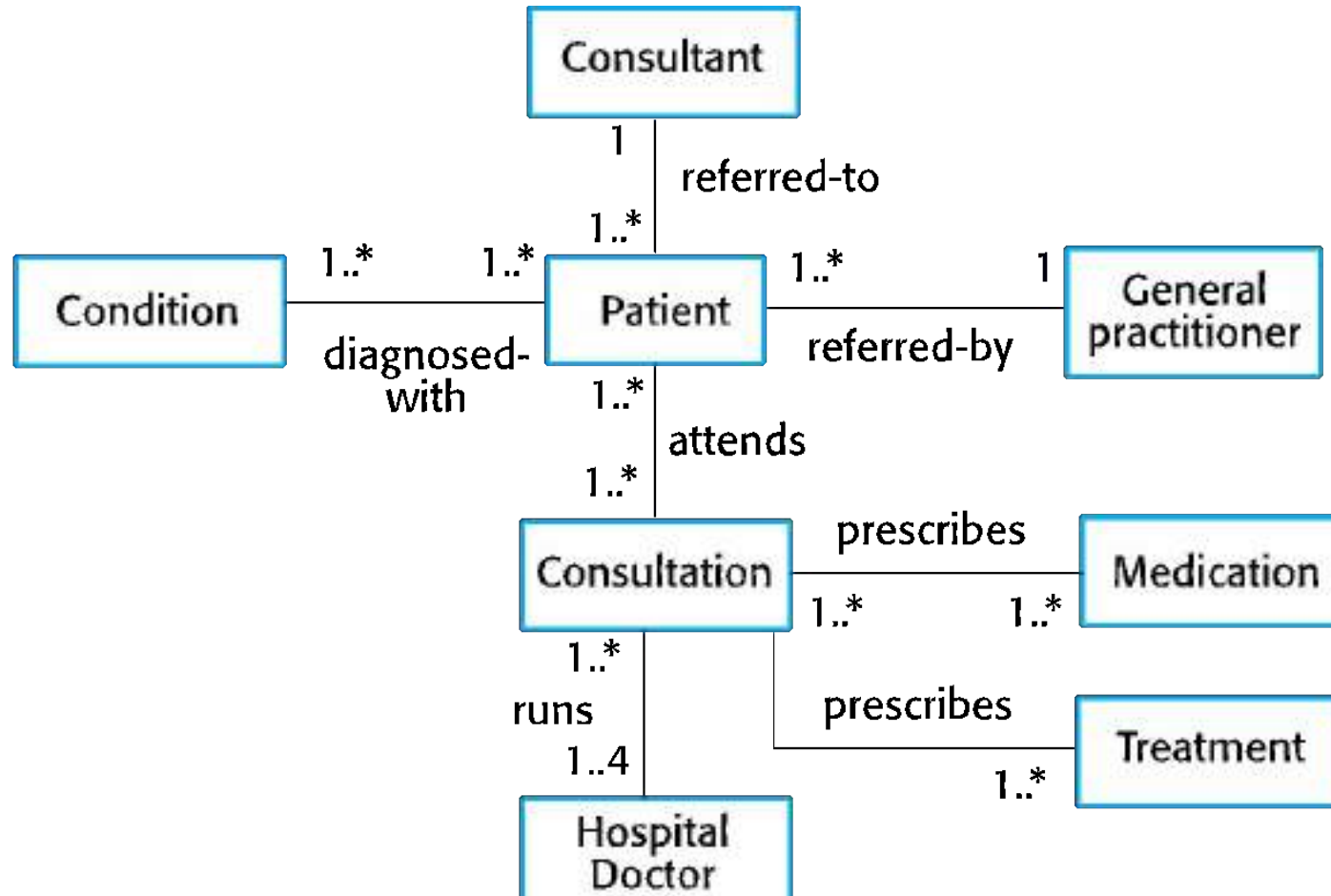


- ✧ Class diagrams are used when developing an object-oriented system model to show the classes in a system and the associations between these classes.
- ✧ An object class can be thought of as a general definition of one kind of system object.
- ✧ An association is a link between classes that indicates that there is some relationship between these classes.
- ✧ When you are developing models during the early stages of the software engineering process, objects represent something in the real world, such as a patient, a prescription, doctor, etc.

UML classes and association



Classes and associations in the MHC-PMS



The Consultation class



Consultation
Doctors Date Time Clinic Reason Medication prescribed Treatment prescribed Voice notes Transcript ...
New () Prescribe () RecordNotes () Transcribe () ...

Generalization



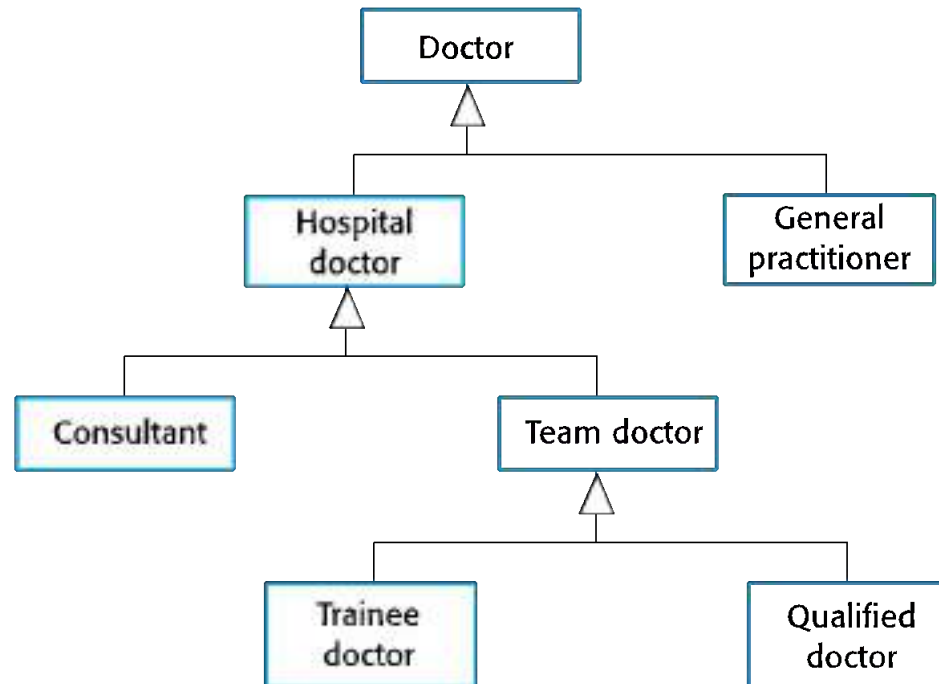
- ✧ Generalization is an everyday technique that we use to manage complexity.
- ✧ Rather than learn the detailed characteristics of every entity that we experience, we place these entities in more general classes (animals, cars, houses, etc.) and learn the characteristics of these classes.
- ✧ This allows us to infer that different members of these classes have some common characteristics e.g. squirrels and rats are rodents.

Generalization

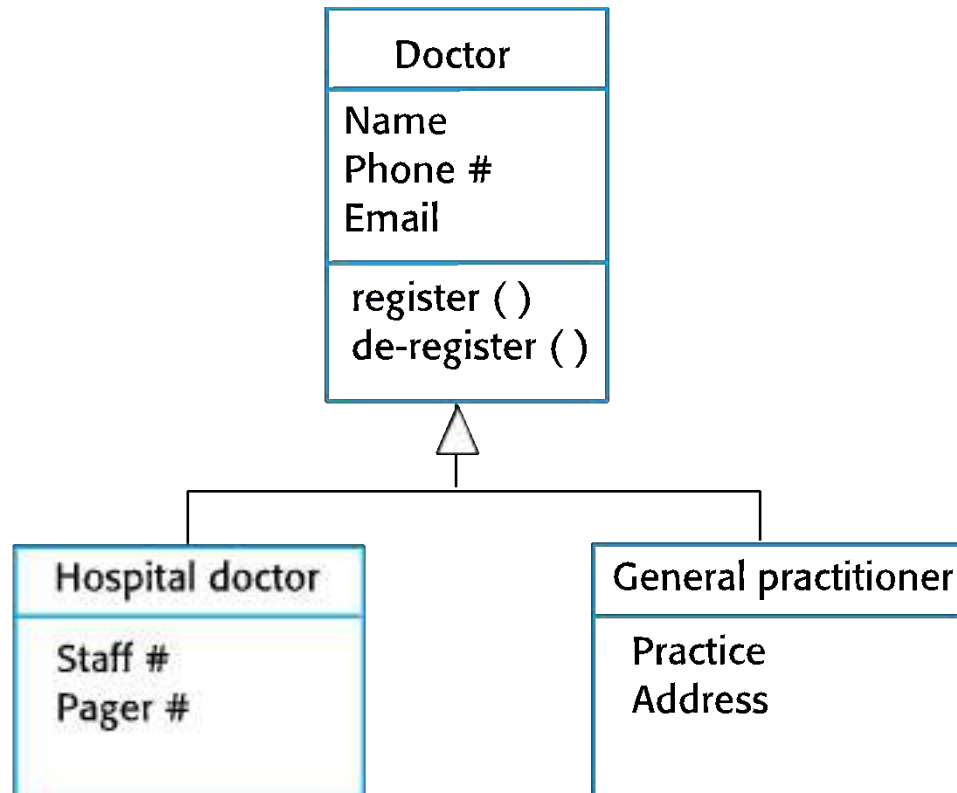


- ✧ In modeling systems, it is often useful to examine the classes in a system to see if there is scope for generalization. If changes are proposed, then you do not have to look at all classes in the system to see if they are affected by the change.
- ✧ In object-oriented languages, such as Java, generalization is implemented using the class inheritance mechanisms built into the language.
- ✧ In a generalization, the attributes and operations associated with higher-level classes are also associated with the lower-level classes.
- ✧ The lower-level classes are subclasses inherit the attributes and operations from their superclasses. These lower-level classes then add more specific attributes and operations.

A generalization hierarchy



A generalization hierarchy with added detail

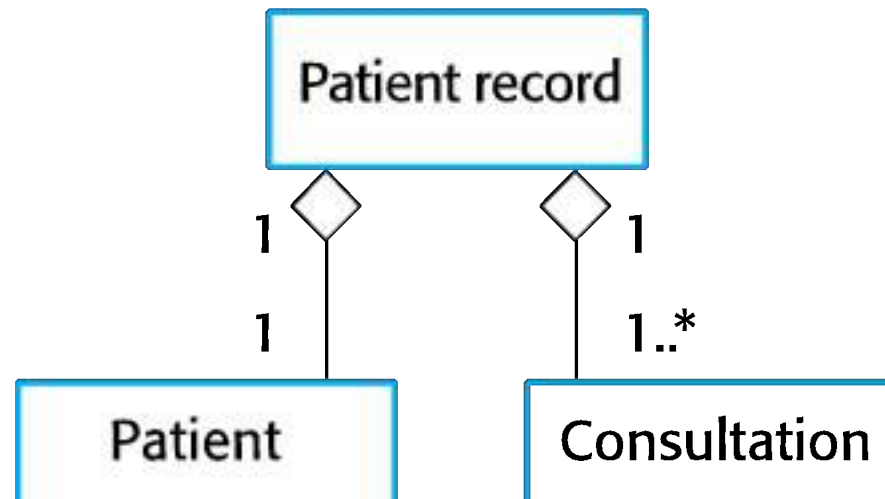


Object class aggregation models



- ✧ An aggregation model shows how classes that are collections are composed of other classes.
- ✧ Aggregation models are similar to the part-of relationship in semantic data models.

The aggregation association





Behavioral models

Behavioral models



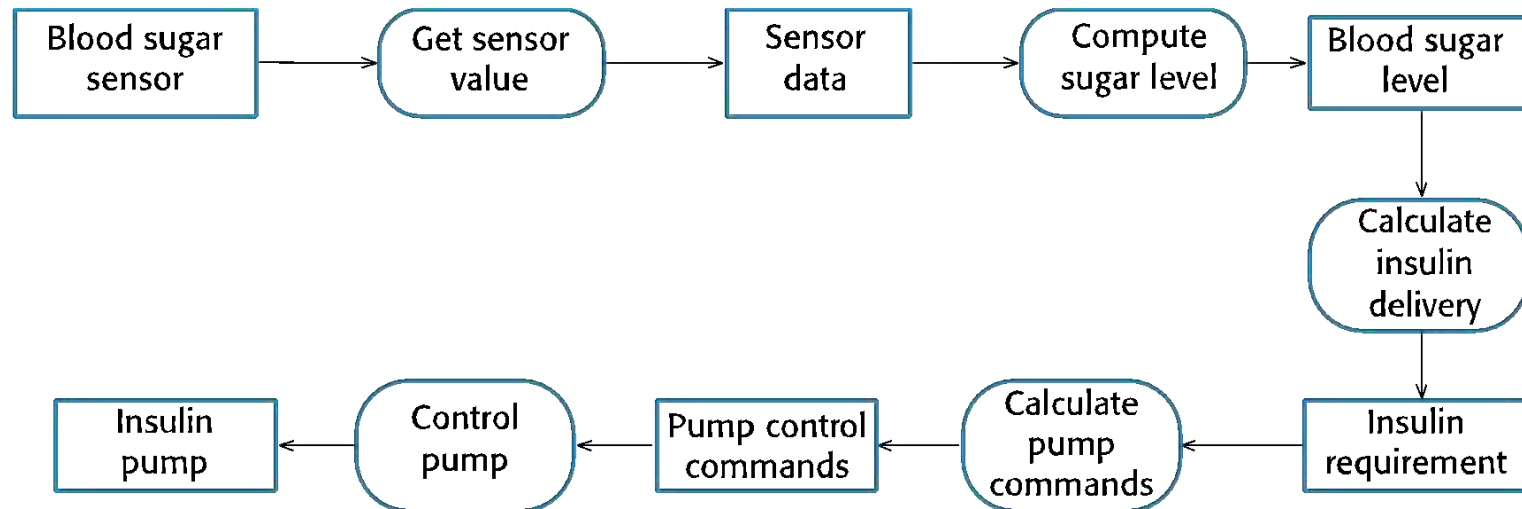
- ✧ Behavioral models are models of the dynamic behavior of a system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment.
- ✧ You can think of these stimuli as being of two types:
 - **Data** Some data arrives that has to be processed by the system.
 - **Events** Some event happens that triggers system processing. Events may have associated data, although this is not always the case.

Data-driven modeling



- ✧ Many business systems are data-processing systems that are primarily driven by data. They are controlled by the data input to the system, with relatively little external event processing.
- ✧ Data-driven models show the sequence of actions involved in processing input data and generating an associated output.
- ✧ They are particularly useful during the analysis of requirements as they can be used to show end-to-end processing in a system.

An activity model of the insulin pump's operation

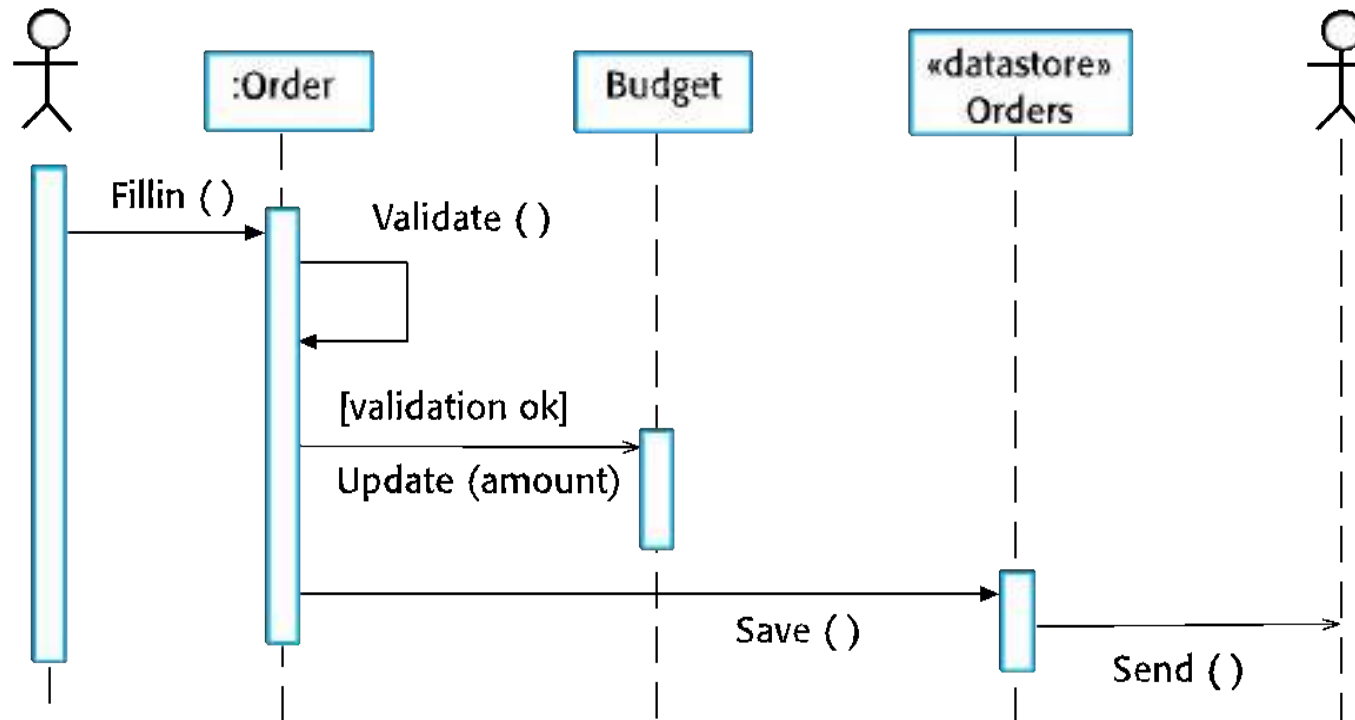


Order processing



Purchase officer

Supplier



Event-driven modeling



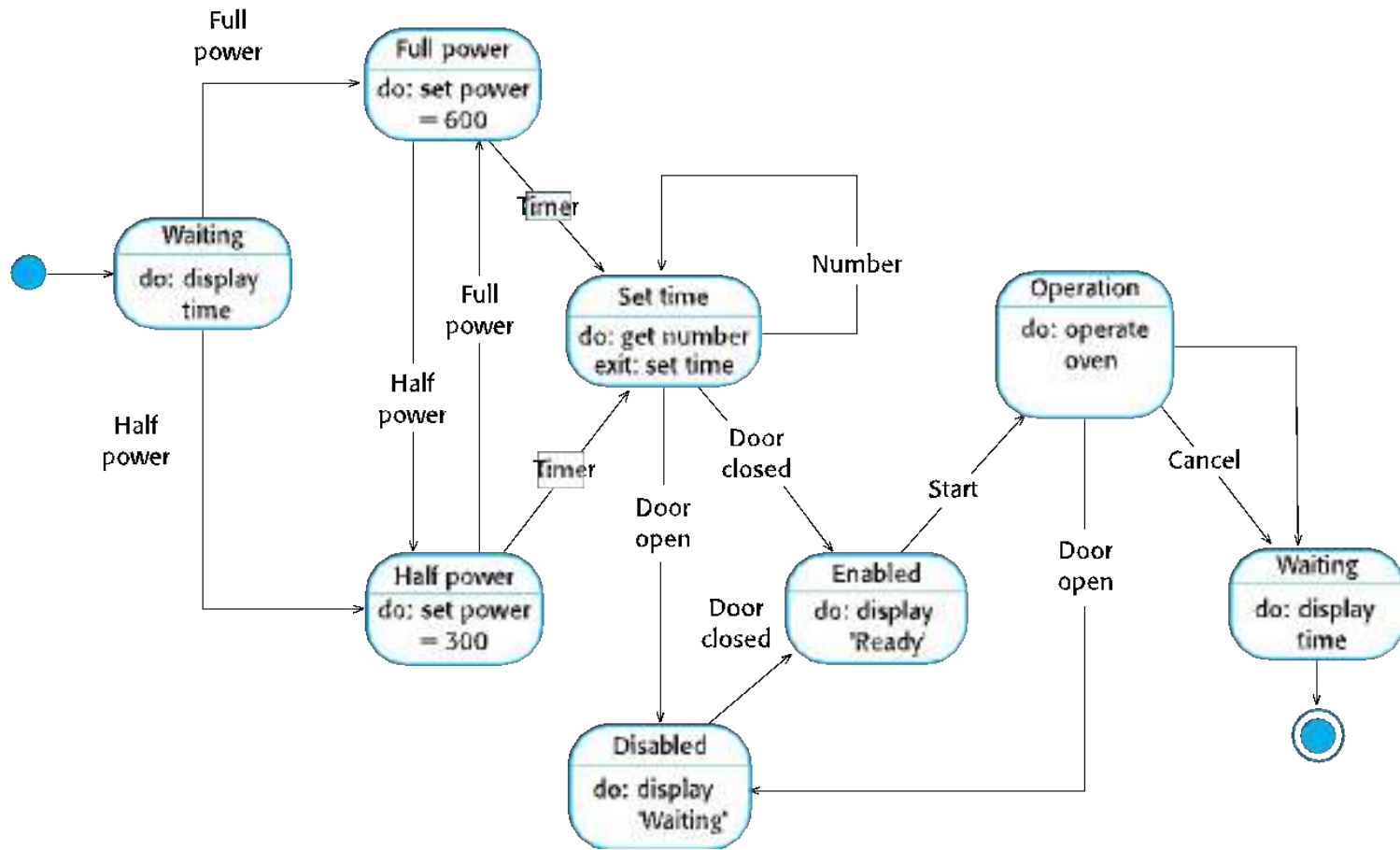
- ✧ Real-time systems are often event-driven, with minimal data processing. For example, a landline phone switching system responds to events such as 'receiver off hook' by generating a dial tone.
- ✧ Event-driven modeling shows how a system responds to external and internal events.
- ✧ It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another.

State machine models

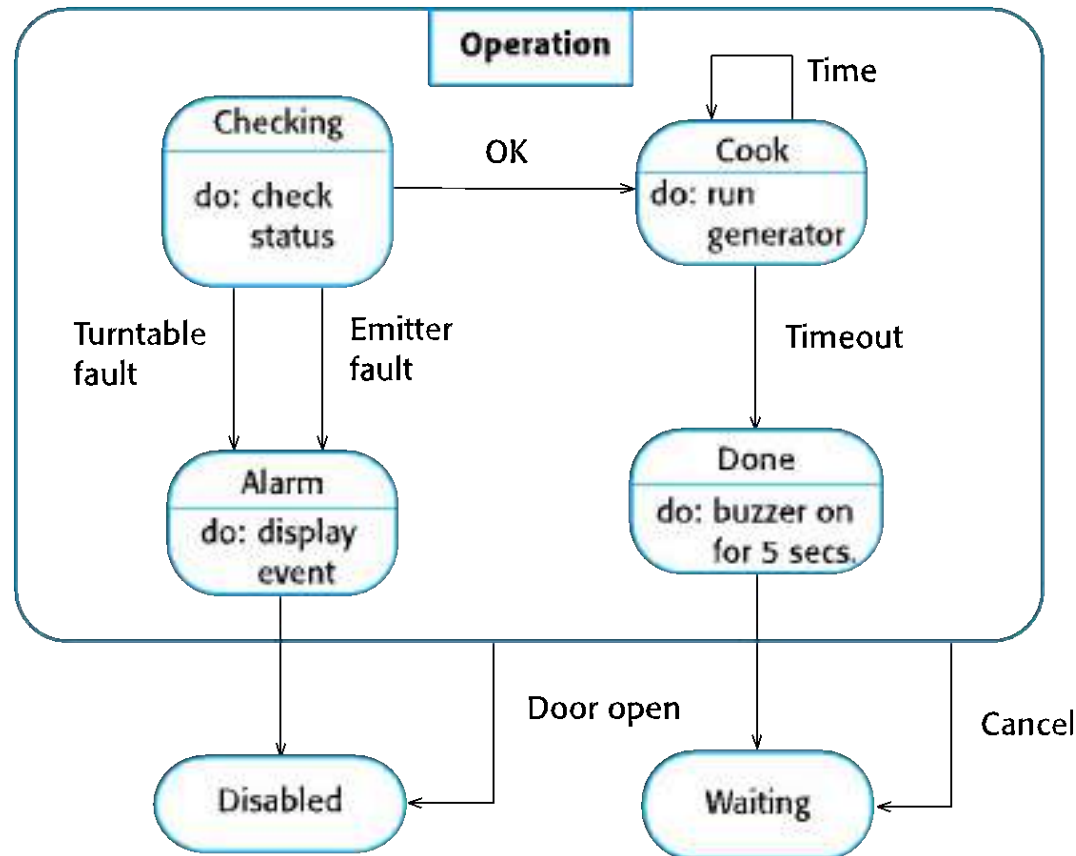


- ✧ These model the behaviour of the system in response to external and internal events.
- ✧ They show the system's responses to stimuli so are often used for modelling real-time systems.
- ✧ State machine models show system states as nodes and events as arcs between these nodes. When an event occurs, the system moves from one state to another.
- ✧ Statecharts are an integral part of the UML and are used to represent state machine models.

State diagram of a microwave oven



Microwave oven operation



States and stimuli for the microwave oven (a)



State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for five seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding.

States and stimuli for the microwave oven (b)



Stimulus	Description
Half power	The user has pressed the half-power button.
Full power	The user has pressed the full-power button.
Timer	The user has pressed one of the timer buttons.
Number	The user has pressed a numeric key.
Door open	The oven door switch is not closed.
Door closed	The oven door switch is closed.
Start	The user has pressed the Start button.
Cancel	The user has pressed the Cancel button.



Model-driven engineering

Model-driven engineering



- ✧ Model-driven engineering (MDE) is an approach to software development where models rather than programs are the principal outputs of the development process.
- ✧ The programs that execute on a hardware/software platform are then generated automatically from the models.
- ✧ Proponents of MDE argue that this raises the level of abstraction in software engineering so that engineers no longer have to be concerned with programming language details or the specifics of execution platforms.

Usage of model-driven engineering



- ✧ Model-driven engineering is still at an early stage of development, and it is unclear whether or not it will have a significant effect on software engineering practice.
- ✧ Pros
 - Allows systems to be considered at higher levels of abstraction
 - Generating code automatically means that it is cheaper to adapt systems to new platforms.
- ✧ Cons
 - Models for abstraction and not necessarily right for implementation.
 - Savings from generating code may be outweighed by the costs of developing translators for new platforms.

Model driven architecture



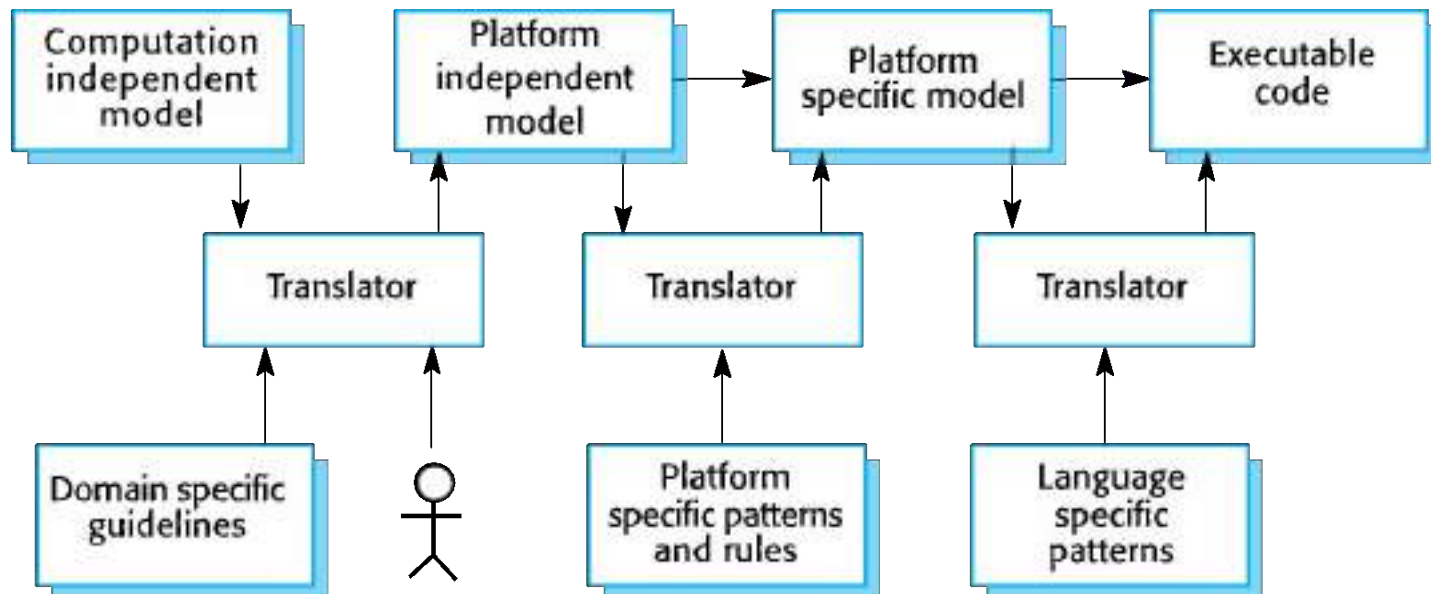
- ✧ Model-driven architecture (MDA) was the precursor of more general model-driven engineering
- ✧ MDA is a model-focused approach to software design and implementation that uses a subset of UML models to describe a system.
- ✧ Models at different levels of abstraction are created. From a high-level, platform independent model, it is possible, in principle, to generate a working program without manual intervention.

Types of model

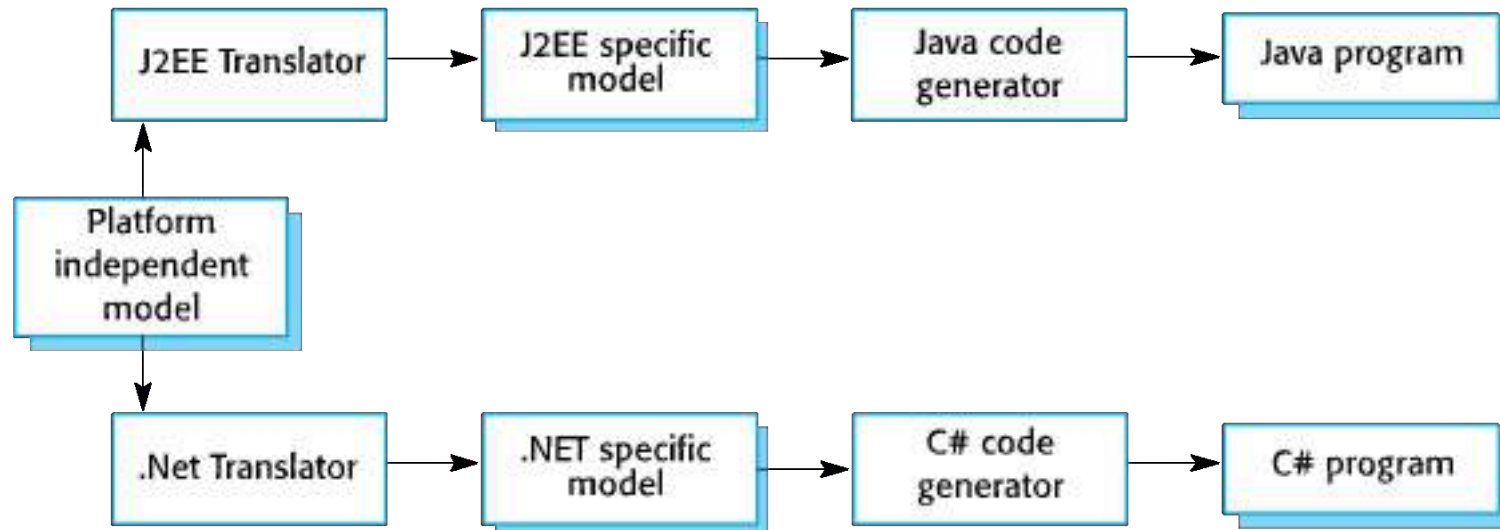


- ✧ A computation independent model (CIM)
 - These model the important domain abstractions used in a system. CIMs are sometimes called domain models.
- ✧ A platform independent model (PIM)
 - These model the operation of the system without reference to its implementation. The PIM is usually described using UML models that show the static system structure and how it responds to external and internal events.
- ✧ Platform specific models (PSM)
 - These are transformations of the platform-independent model with a separate PSM for each application platform. In principle, there may be layers of PSM, with each layer adding some platform-specific detail.

MDA transformations



Multiple platform-specific models



Agile methods and MDA



- ✧ The developers of MDA claim that it is intended to support an iterative approach to development and so can be used within agile methods.
- ✧ The notion of extensive up-front modeling contradicts the fundamental ideas in the agile manifesto and I suspect that few agile developers feel comfortable with model-driven engineering.
- ✧ If transformations can be completely automated and a complete program generated from a PIM, then, in principle, MDA could be used in an agile development process as no separate coding would be required.

Adoption of MDA



- ✧ A range of factors has limited the adoption of MDE/MDA
- ✧ Specialized tool support is required to convert models from one level to another
- ✧ There is limited tool availability and organizations may require tool adaptation and customisation to their environment
- ✧ For the long-lifetime systems developed using MDA, companies are reluctant to develop their own tools or rely on small companies that may go out of business

Adoption of MDA



- ✧ Models are a good way of facilitating discussions about a software design. However the abstractions that are useful for discussions may not be the right abstractions for implementation.
- ✧ For most complex systems, implementation is not the major problem – requirements engineering, security and dependability, integration with legacy systems and testing are all more significant.

Adoption of MDA



- ✧ The arguments for platform-independence are only valid for large, long-lifetime systems. For software products and information systems, the savings from the use of MDA are likely to be outweighed by the costs of its introduction and tooling.
- ✧ The widespread adoption of agile methods over the same period that MDA was evolving has diverted attention away from model-driven approaches.

Key points



- ✧ A model is an abstract view of a system that ignores system details. Complementary system models can be developed to show the system's context, interactions, structure and behavior.
- ✧ Context models show how a system that is being modeled is positioned in an environment with other systems and processes.
- ✧ Use case diagrams and sequence diagrams are used to describe the interactions between users and systems in the system being designed. Use cases describe interactions between a system and external actors; sequence diagrams add more information to these by showing interactions between system objects.
- ✧ Structural models show the organization and architecture of a system. Class diagrams are used to define the static structure of classes in a system and their associations.

Key points



- ✧ Behavioral models are used to describe the dynamic behavior of an executing system. This behavior can be modeled from the perspective of the data processed by the system, or by the events that stimulate responses from a system.
- ✧ Activity diagrams may be used to model the processing of data, where each activity represents one process step.
- ✧ State diagrams are used to model a system's behavior in response to internal or external events.
- ✧ Model-driven engineering is an approach to software development in which a system is represented as a set of models that can be automatically transformed to executable code.